



FORMATION PROMETHEUS & GRAFANA

Guide stagiaire — Jour 3, corrigés

Les réponses du jour 3, version pratique

Sommaire

Les réponses du jour 3, version pratique	2
Comment utiliser ce corrigé	2
Échauffement — les quatre signaux dans Explore	2
TP A — La boutique en quatre signaux dorés	2
TP B — Le serveur en méthode USE	3
TP C — Prometheus, Alertmanager, Teams, Slack et e-mail	3
TP D — L'alerting de Grafana, et la comparaison	4
Escape game — les sept sabotages possibles	5
Les questions éclair	5

Les réponses du jour 3, version pratique

Comment utiliser ce corrigé

Ce document donne les réponses à toutes les questions du guide du jour 3. Les **requêtes** de chaque panel sont dans l'annexe « Requêtes de secours » du guide lui-même. Lisez-le **après** avoir cherché : c'est en se trompant dans le lab qu'on apprend, et ça ne casse rien.

Échauffement — les quatre signaux dans Explore

Signal	Requête possible (pour toute la boutique)	Valeur attendue	En une phrase
Trafic	<code>sum(rate(http_requests_total{job="shop-api"}[5m]))</code>	6 à 7 req/s	le nombre de requêtes reçues par seconde, toutes instances et routes confondues
Erreurs	<code>sum(rate(http_requests_total{job="shop-api", status=~"5.."}[5m])) / sum(rate(http_requests_total{job="shop-api"}[5m]))</code>	0 sans chaos	la part des requêtes qui finissent en erreur 5xx
Latence p95	<code>histogram_quantile(0.95, sum by (le) (rate(http_request_duration_seconds_bucket{job="shop-api"}[5m])))</code>	0,15 à 0,25 s	95 % des requêtes sont plus rapides que cette durée
Saturation	<code>100 * (1 - avg(rate(node_cpu_seconds_total{mode="idle"}[5m])))</code>	quelques %	la part du temps où les processeurs de la machine travaillent

Les erreurs fréquentes de l'IA : le compteur brut sans `rate` (des milliers qui montent), `status="500"` au lieu de `status=~"5.."` (oublie les 502, 503...), un débit d'erreurs au lieu d'un taux (pas de division), la moyenne `_sum / _count` au lieu du p95, `sum by (route)` sans `le` (« No data »), le CPU **libre** au lieu du CPU utilisé (oubli du `1 -`).

TP A — La boutique en quatre signaux dorés

Étape 1 — Le signal qui a réagi le plus lentement. Les erreurs et la latence. Elles sont calculées avec `rate()` sur une fenêtre (`$_rate_interval`, au moins une minute) : la fenêtre contient encore des minutes saines au début de la panne, la courbe monte progressivement. Le CPU, lui, réagit dès le

scrape suivant. Une fenêtre courte réagit vite mais « clignote », une fenêtre longue est stable mais lente : c'est le même compromis que le `for` des alertes.

Étape 3 — Pourquoi `rate` pour un montant ? `shop_revenue_euros_total` est un **compteur** : tout ce qui a été vendu depuis le démarrage de l'application, remis à zéro à chaque redémarrage. Sa valeur brute ne dit rien de la situation actuelle. `rate()` donne la **vitesse** en euros par seconde, $\times 3600$ en euros par heure. Le compteur kilométrique contre le compteur de vitesse. Autre réponse juste :

`increase(...[1h])` , « combien sur la dernière heure ».

Étape 5 — Le crash test. Avec `./lab.sh traffic 30` , le trafic, la saturation et le chiffre d'affaires montent ; les erreurs et la latence restent vertes. **Ce n'est pas une panne** : c'est un afflux de clients qui se passe bien. C'est pour ça que la case Trafic est bleue et jamais rouge : on ne réveille personne pour une bonne nouvelle.

TP B — Le serveur en méthode USE

Le piège du panel 2. `node_load1 / count(node_cpu_seconds_total{mode="idle"})` renvoie « No data » : à gauche, une série avec les labels `instance` et `job` ; à droite, un `count` sans aucun label. PromQL associe les séries qui ont **les mêmes labels** : aucune ne correspond. La correction : `node_load1 / on (instance) count by (instance) (node_cpu_seconds_total{mode="idle"})` (on garde `instance` des deux côtés et on dit sur quoi associer). Une charge divisée par le nombre de cœurs : 1 = plein, 2 = la moitié du travail attend.

MemAvailable et pas MemFree . Linux utilise la mémoire libre comme cache disque et la rend quand on en a besoin. `MemFree` est toujours bas et fait peur pour rien ; `MemAvailable` dit ce qui est réellement disponible.

Le dashboard 1860 ou le vôtre ? À 3 h du matin, le vôtre : dix panels, des couleurs, la réponse à « ça va ? » en cinq secondes. Pour une enquête de deux heures, le 1860 : il montre tout. On commence souvent par un dashboard de la communauté, et on construit son propre dashboard de synthèse au-dessus.

TP C — Prometheus, Alertmanager, Teams, Slack et e-mail

Partie 1 — Les règles.

1. **Symptômes** (les clients le sentent) : `ShopHighErrorRate` , `ShopCheckoutSlow` , `BlackboxProbeFailed` , et `TargetDown` (on ne mesure plus rien). **Cause** : `HostHighCpuLoad` : un CPU haut peut être un serveur qui travaille bien.
2. `for: 1m` : la condition doit rester vraie une minute avant de notifier (*Pending* → *Firing*) ; c'est l'anti-faux positif. Le label `team` ne sert qu'au **roulage** dans l'Alertmanager.
3. Réveillent quelqu'un (critical) : `ShopHighErrorRate` , `TargetDown` , `BlackboxProbeFailed` . Ne doit jamais réveiller : `HostHighCpuLoad` (warning).

Partie 2 — L'arbre de roulage.

Labels de l'alerte	Receivers
<code>severity=critical team=boutique</code>	<code>astreinte-teams</code> et boutique (grâce à <code>continue: true</code>)
<code>severity=warning team=boutique</code>	boutique (Slack + e-mail)
<code>severity=warning team=infra</code>	infra-inbox
<code>severity=critical</code> (sans team)	<code>astreinte-teams</code>
<code>severity=info team=logistique</code>	<code>inbox-default</code> (aucune branche ne correspond : la racine)

Sans `continue: true`, la première ligne ne donne plus que `astreinte-teams` : l'astreinte est prévenue, mais pas l'équipe boutique. C'est l'erreur de routage la plus fréquente.

Partie 3 — La chronologie. *Pending* 15 à 30 s après le chaos (le temps d'un scrape et d'une évaluation) ; *Firing* une minute plus tard (le `for`) ; notifications 10 s après (`group_wait`). Une carte Teams (astreinte) et un message Slack (boutique) dans l'Inbox, un e-mail dans Mailpit (boutique). Les deux instances sont dans **le même** message : `group_by: [alertname, job]`. Le *resolved* arrive jusqu'à une minute après la fin du chaos (`group_interval: 1m`).

Partie 4 — L'inhibition. `ShopCheckoutSlow` existe, mais elle est *suppressed* : une alerte critique de la même équipe est active. Elle n'envoie rien. L'inhibition ne joue que si l'alerte « source » est déjà active : c'est pour ça qu'on lance les erreurs d'abord.

Partie 5 — Silence ou inhibition ? L'inhibition est une **règle permanente** de la configuration ; le silence est une **décision humaine**, ponctuelle, avec un auteur, une raison et une durée.

Partie 6 — Pourquoi l'adresse Teams dans un fichier ? C'est un secret : quiconque l'a peut écrire dans le canal. Le YAML part dans Git ; le fichier `alertmanager/secrets/teams_url`, non.

TP D — L'alerting de Grafana, et la comparaison

La règle métier. Avec `./lab.sh traffic 1`, le chiffre d'affaires passe d'environ 180 000 € par heure à moins de 40 000 en deux à trois minutes : la règle passe *Pending* puis *Firing*, et l'e-mail arrive dans Mailpit (politique `team = commerce`).

	Prometheus + Alertmanager	Grafana
Où vit la configuration ?	des fichiers YAML	la base de Grafana (ou des fichiers de provisioning)
Comment la versionner et la relire ?	Git, merge request : naturellement	export, provisioning, Git Sync : possible, moins naturel
Comment la tester avant de la déployer ?	<code>promtool test rules</code> , <code>amtool config routes test</code>	<i>Preview</i> de la règle, <i>Test</i> du contact point
Qui peut la modifier ?	ceux qui touchent au YAML	quiconque a les droits sur le dossier
Plusieurs sources (SQL, logs) ?	non, les métriques Prometheus	oui, toute source de données Grafana

	Prometheus + Alertmanager	Grafana
Si Grafana tombe ?	les alertes continuent	plus d'alertes
Pour quelles alertes chez moi ?	les alertes techniques, critiques, qui doivent survivre à tout	les alertes métier, multi-sources, réglables par les équipes

Les deux coexistent très bien. La seule règle : **jamais la même alerte des deux côtés.**

Escape game — les sept sabotages possibles

Votre Codespace en a tiré deux au hasard (`./jour3.sh solution` vous a dit lesquels).

N°	Sabotage	Où ça se voit	Alerte et canal
1	Latence ×10 sur <code>shop-api-1</code> seulement	case Latence, heatmap ; variable <code>instance</code> sur <code>shop-api-1</code> pour le voir nettement	<code>ShopCheckoutSlow</code> (warning) → Slack + e-mail ; <i>inhibée</i> si l'incident 2 est aussi en cours
2	40 % d'erreurs sur <code>shop-api-2</code> seulement	case Erreurs (15-20 % au global, 30-40 % sur <code>shop-api-2</code>)	<code>ShopHighErrorRate</code> sur <code>shop-api-2</code> → Teams + Slack + e-mail
3	Redis arrêté	Explore : <code>redis_up</code> = 0 (alors que <code>up{job="redis"}</code> reste à 1 : c'est l'exporter qui répond)	aucune : l'alerte qui manque, c'est <code>redis_up == 0</code>
4	Trafic ×5	cases Trafic et Saturation, chiffre d'affaires en hausse	éventuellement <code>HostHighCpuLoad</code> → infra. Pas une panne
5	CPU saturé pendant 15 minutes	case Saturation, dashboard USE (CPU, charge, pression)	<code>HostHighCpuLoad</code> (warning) → infra-inbox. La boutique reste rapide : une saturation n'est pas encore une panne
6	<code>shop-api-1</code> arrêtée	State timeline, <code>up</code> = 0, <code>probe_success</code> = 0 ; trafic mesuré et chiffre d'affaires divisés par deux (il n'y a pas de répartiteur de charge)	<code>TargetDown</code> → Teams + Slack + e-mail ; <code>BlackboxProbeFailed</code> → Teams
7	Fuite mémoire	Explore : <code>process_resident_memory_bytes{job="shop-api"}</code> qui grimpe	aucune : l'alerte qui manque, une <code>predict_linear</code> sur la mémoire du processus

La leçon des incidents 3 et 7 : un dashboard ne montre que ce qu'on a pensé à y mettre, et une panne sans alerte, c'est une alerte à écrire. C'est le travail d'une revue post-incident.

Les questions éclair

1. `http_requests_total` est un compteur. Il ne fait que monter depuis le démarrage et repart à zéro au redémarrage : on lit sa vitesse avec `rate()` , jamais sa valeur brute.

2. **Le p95 plutôt que la moyenne** : la moyenne ne décrit personne (95 clients à 50 ms et 5 à 10 s donnent 550 ms) ; le p95 est une promesse qu'on peut tenir pour 95 % des clients.
3. **Le for** : la condition doit tenir cette durée avant de notifier. C'est l'anti-faux positif.
4. **continue: true** : l'alerte continue de descendre l'arbre après une branche qui lui correspond ; plusieurs receivers la reçoivent.
5. **Inhibition** : règle permanente de la configuration. **Silence** : décision humaine, ponctuelle, avec un auteur et une durée.
6. **L'adresse Teams dans un fichier** : c'est un secret, et le YAML part dans Git.
7. **MemAvailable** : Linux utilise la mémoire libre comme cache ; **MemFree** est toujours bas.
8. **Une requête SQL** : Grafana. **Survivre à une panne de Grafana** : Prometheus et Alertmanager.